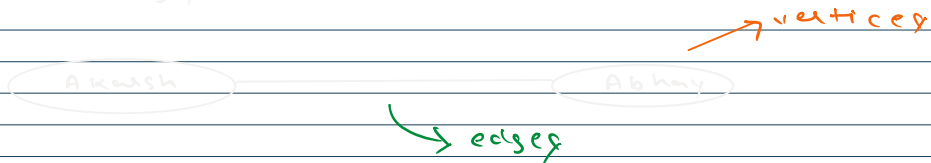
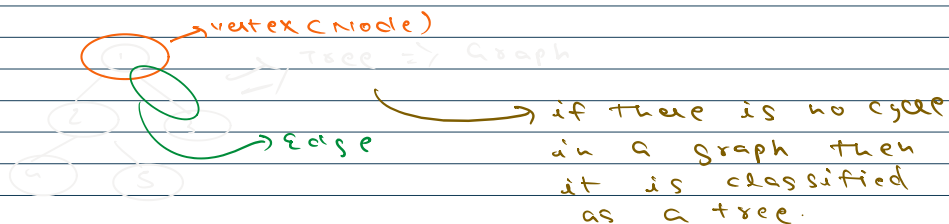


Graphs

A graph is a finite collection of vertices (called nodes) and edges



vertex is a point where value/data is stored.
 edge is a line/link that connect two vertices



"all trees are graphs, but not all graphs are trees".

Google maps

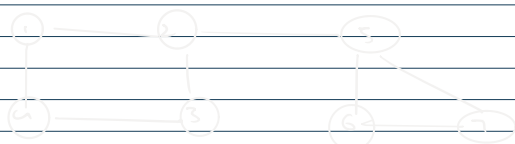
Types of Graphs



Digraph

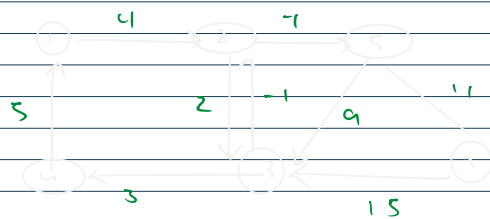
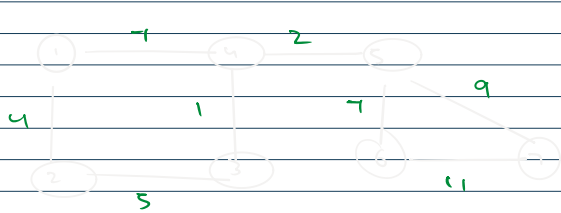


Undirected graph



Weighted Graphs



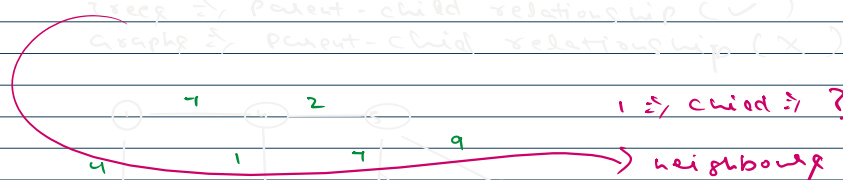


How can we represent these graphs



parent $\Rightarrow$ Binary tree
child1 child2

Tree $\Rightarrow$ Parent-child relationship ($\checkmark$)
Graphs $\Rightarrow$ Parent-child relationship ($\times$)



1 $\Rightarrow$ child $\Rightarrow$?

neighbour

- 1 $\Rightarrow$ [4 | 7], [2 | 4]
- 2 $\Rightarrow$ [1 | 4], [3 | 5]
- 3 $\Rightarrow$ [2 | 5], [4 | 1]
- 5 $\Rightarrow$ [4 | 2], [6 | 7], [7 | 9]

11. Adjacency Matrix

	1	2	3	4	5	6	7
1	0	4	0	7	0	0	0
2	4	0	5	0	0	0	0
3	0	5	0	1	0	0	0
4	7	0	1	0	2	0	0
5	0	0	0	2	0	7	9
6	0	0	0	0	7	0	11
7	0	0	0	0	9	11	0

Comp $\rightarrow$ Space wastage
Resistant data

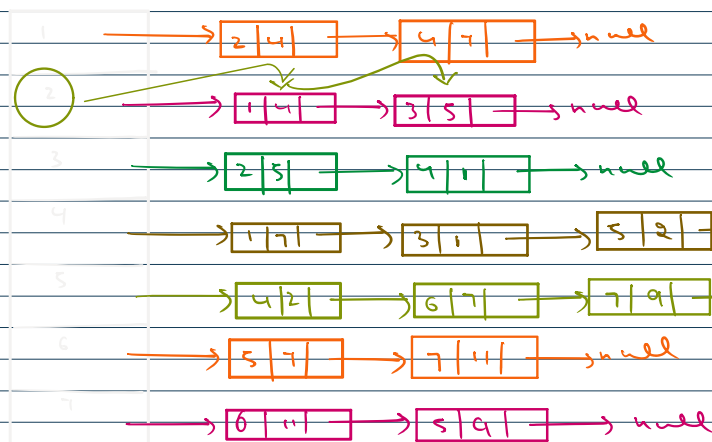
Prop: 11-Search time is slow.
21. works well in graph search

1-7 $\Rightarrow$ Rel $\Rightarrow$? $> 0 \Rightarrow$ yes

$\rightarrow$ No

Space $\Rightarrow O(V \times V)$

13. Adjacency List

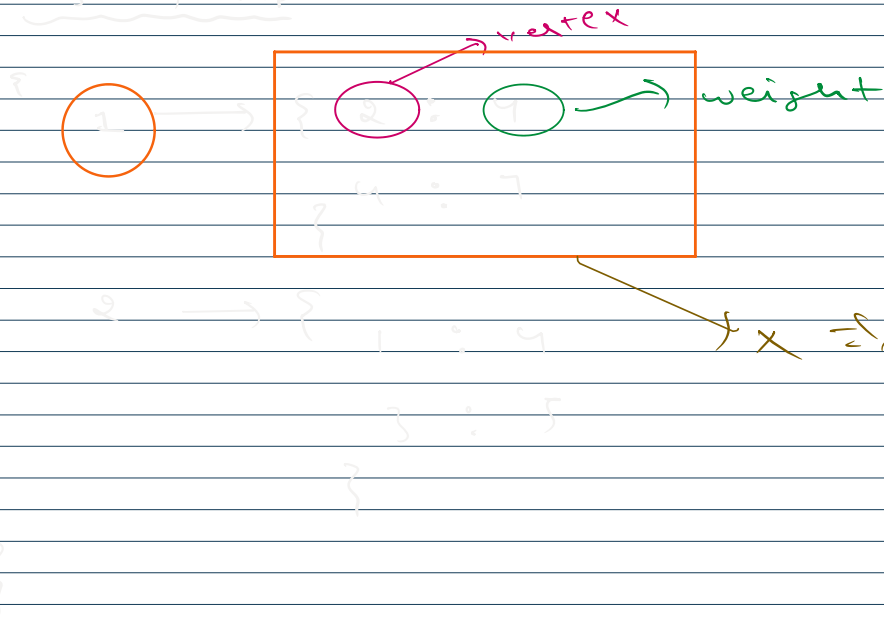


Space $\Rightarrow O(V+E)$

2-3 $\Rightarrow$ rel

$\downarrow$
?

31. Adjacency map



1 - 7
 $\downarrow \downarrow$
 sel. ≥ 1 ?

$\times \nless, sel. \Rightarrow no.$

(1) - (2)

complete graph



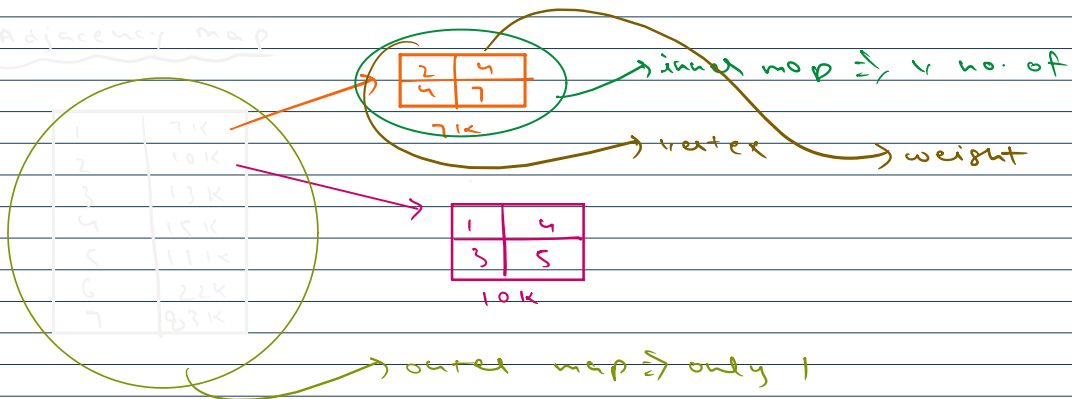
$v = 4$
 $e = ? \Rightarrow 6 \Rightarrow formula$



$v = 3$
 $e = 3$

$$\left. \begin{array}{l} 1 \leftrightarrow 2 \quad \{ (v-1) \\ 2 \leftrightarrow 3 \quad \{ (v-1) \\ 3 \leftrightarrow 4 \quad \{ (v-1) \\ 4 \leftrightarrow 1 \quad \{ (v-1) \end{array} \right\} \frac{v \cdot (v-1)}{2}$$

Adjacency map



Java-DSA-2.0 Day 41 src Graph

```
import java.util.HashMap;

public class Graph {
    private HashMap<Integer, HashMap<Integer, Integer>> map;

    // map => key -> vertices, value -> innerMap
    // innerMap => key -> connected vertex, value -> weight of edge

    public Graph(int v) {
        map = new HashMap<>();
        for (int i = 1; i <= v; i++) {
            map.put(i, new HashMap<>());
        }
    }

    public void addEdge(int v1, int v2, int cost) {
        HashMap<Integer, Integer> innerMap1 = map.get(v1); // 5k
        innerMap1.put(v2, cost);

        HashMap<Integer, Integer> innerMap2 = map.get(v2); // 7k
        innerMap2.put(v1, cost);
    }
}
```

Handwritten diagrams illustrating the graph structure and the map representation:

- A graph with 7 vertices (1-7) and edges with weights: (1,2)=7, (1,5)=2, (2,3)=5, (3,4)=1, (4,5)=2, (5,6)=7, (5,7)=9, (6,7)=11.
- A table representing the map structure:

Vertex	Connected Vertex	Weight
1	5	2
2	1	7
3	2	5
4	3	1
5	1	2
5	6	7
6	5	7
7	5	9
7	6	11

Additional diagrams show the inner maps for vertices 1, 2, 3, 4, 5, 6, and 7, each represented as a 2x2 grid. Arrows indicate the mapping from the table to these inner maps.

Talking Coding Blocks...